Lab: Flexbox

In a previous lab we discussed web layout using CSS grids. CSS grids allow you to turn HTML containers into rows and columns. Another method of layout in CSS is called ***flexbox***. Like the term indicates it allows for layouts that are flexible. This means items can grow or shrink, depending on available space or can distribute themselves evenly based on available space.

After your great work with Tommy's Toy, Tommy recommended you to his brother Tim. Tim runs a travel agency called Tim's Travel Agency. He wants your assistance in the layout out his website as well, for this lab we will use CSS flexbox as an alternative method to layout a webpage.

Setup

Download *CSS-Flexbox.zip* and unzip it. It should contain a HTML document called *travel-agency.html* and includes all of the related images. Load *travel-agency.html* into a code editor.

Take a moment to look over the HTML file. The website is broken into sections such as navigation, international travel, US travel, etc. All of the *<section>* tags are encapsulated within a *.container* div.

Attached to the HTML document is a stylesheet that is laying out the the basic formatting including typography, colors and basic layouts such as width and heights.

Flexbox Basics

So what exactly is a CSS flexbox? Flexbox is a method in CSS where a HTML container is turned into a flexible box, hence the phrase ***flexbox***. Once a HTML container is turned into a flexbox the direct HTML children of the container become flex items. Once you have flex items the parent flexbox itself can control where the flex items go, including horizontal location or whether the items are distributed based on the available space or not.

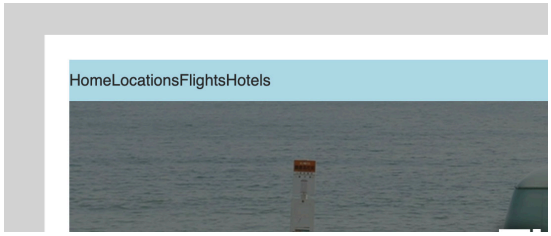
© David Hurwich

Navigation Bar

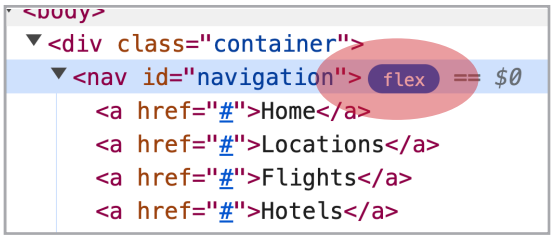
Let's between by laying out the navigation links in the *<nav>* tag.

Let's turn the *<nav>* tag into a flexbox using CSS. Type the following:

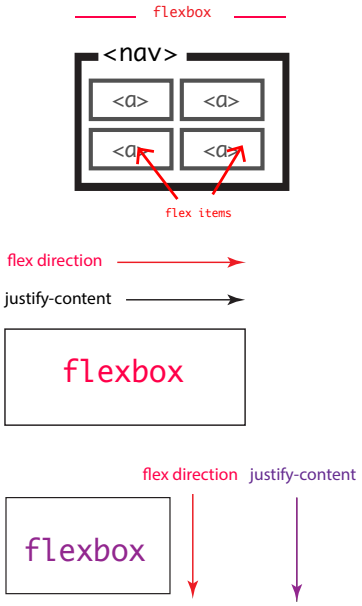
```
#navigation {
  display: flex;
}
```



Save and preview the webpage. At moment it does not look like anything has happened. If you are using Google Chrome or Mozilla Firefox right click and choose *Inspect*. You will see in the *Elements* tab that the *<nav>* tag now has a *"flex"* rounded marker next to it. If you click on the marker it will show you the results of the

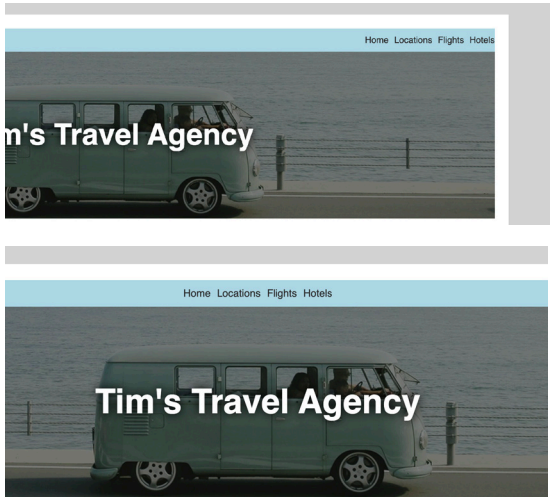


Now that the *<nav>* tag is a flexbox each of the *<a>* tags is considered a flex item. The next step is deciding where the flex items go within the flex box. Flexbox have directions, they can either be rows, where items defaulty lay out horizontally, or they can be columns, which means items defaultly lay out vertically. By default flexbox are rows, meaning the items will display in a horizontal row one after another.



There is another CSS property called *justify-content*. The *justify-content* means how flex items will be placed. The *justify-content* goes along with the direction of the flexbox, whether it is a row or column. If it a row the start of the justify-content is on the left side of the container and the end of justify-content is on the right side. If the flexbox is a column justify-content start is at the top and justify-content end is at the bottom.

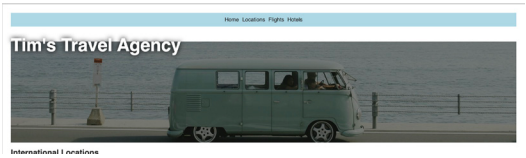
Set *justify-content* to *start*. Save and preview, you will see that the items stay on the far left, as that is the *"start"* or *flex-start*. Change the view to *"end"* , save and preview again. The items should now appear at the far right as that is the end of the flexbox. We can also place the items at the horizontal center of the flexbox by setting the justify-content to center. Set justify-content to *"center"*. It should appear like the image on the right.



© David Hurwich

Gap

At the moment our navigation links are really close together, it would help if they were spaced out a bit from one another. Flex items can be spaced out using the CSS property *"gap"*. Add the following code:

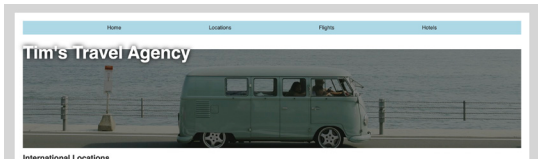



```
#navigation {
   display: flex;
   justifty-content: center;
   gap: 10px;
}
```

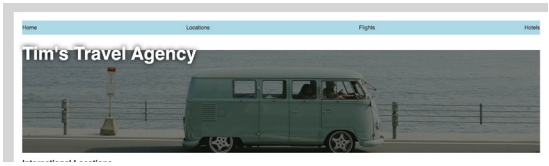
This will put 10px of gap after each item. If the flexbox is a *row* the gap comes horizontally after an item, if the flexbox is a *column* the gap comes vertically after an item.

Distribution

In addition to placing all of the flex items at one spot on the webpage we can also distribute them equally among the available horizontal space. To do this we can use *space-between*, *space-around* and *space-evenly*.

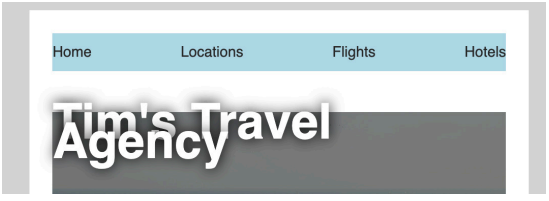


Space evenly will take the available items and place the available evenly on the left and right side of each item. Set *justify-content* to *space-evenly*. Save and preview. You'll see that the navigation items have equally spacing on the left and right side. Readjust the width of the webpage and you'll see that the items readjust as well.



Another option is *space-around*. Space-around sets equal spacing between the 2nd and 2nd to last flex item, but the first and last get 1/2 the normal space on the left and right respectively.

What is we want the items to align flush to the far left or start and the far right or end of the flexbox? The answer is using the value *space-between*. Space-between places the item flex item flush to the start, the last item flush to the end and divides the remaining space evenly. Set justify-content to *space-between*. Readjust the webpage to see that the home and hotels stay flush to their sides and the other items readjust accordingly.



Add Flex Items

What's nice about flexbox is it's ability to be flexible. This means if we later add an item, everything will automcatically adjust. After viewing our progress Tim mentioned he wants another navigational link related to car rentals. Go into the webpage and uncomment this option.

Since this hyperlink is child of the *<nav>* it will also be added as a flexitem. Since the flexbox is set to *space-between* it will added to

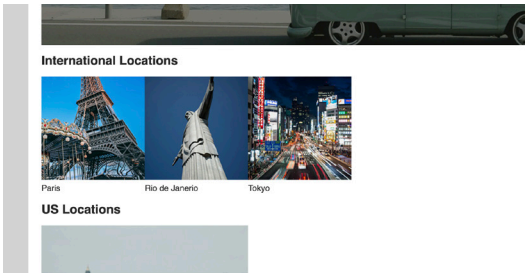
© David Hurwich

the distribution.



Save and view that the all of the navigation links have automatically re-distributed.

International Locations



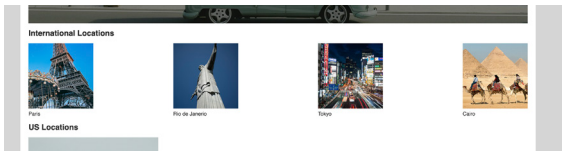
Tim plans to layout out a row of international travel destinations on the website. The idea to have photos and related captions that the users can talk to Tim about in person. In the HTML you'll see a called *#international-international*. Inside are *<figure>* tags combining an image with the location. We want to be able to easy distribute these *<figure>* regardless of how wide the webpage is. This would a good job for flexbox.

Add the following code:

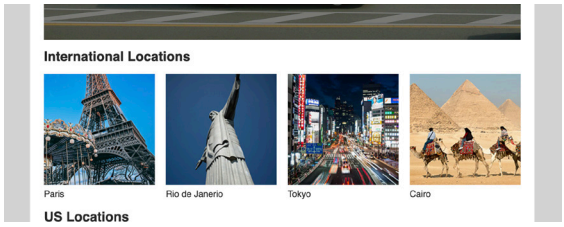
```
#international-layout {
   display: flex;
   justify-content: space-between;
}
```



Like with the navigation bar we are turning the *#international-layout* div into a flexbox. This will make each *<figure>* into a flex item as they are direct children of the *#international-layout* div. The *justify-content: space-between* will take the first *<figure>* which is Paris and place it on the far left. It will then place the last *<figure>* Tokyo and place it on the far right. The remaining item gets equally spacing on the left and right.



After chatting with Tim, he mentioned he forgot to include Cairo in the list of options and would like you to include it. Uncomment the Cairo *<figure>* in the HTML. Save and you will see that all of the <figure> tags readjust their spacing! Flexbox makes it easy to add and subtract content while maintaining a certain spacing style.



© David Hurwich

Splash Area

At the top of the webpage Tim has a splash area with the photo of camper van. He wants the company name to display in the vertical and horizontal center on top of the splash image.

In the HTML find the <section> marked *#splash*. Note that it is a <section> tag with an <h1> inside.

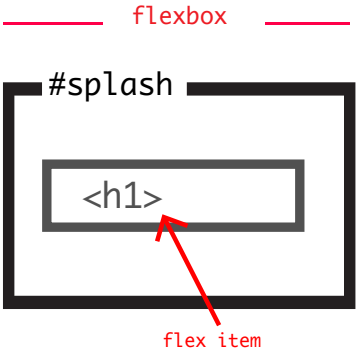
Add the following code:

```
#splash {

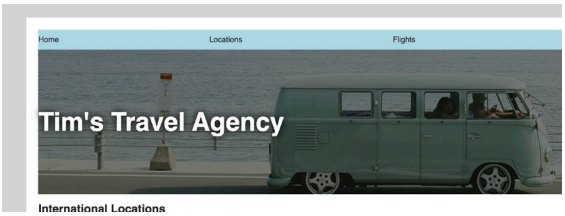
   display: flex;
   flex-direction: column;
   justify-content: center;

}
```

What is occuring here is that we are making the *#splash* area into a flexbox. This would make the <h1> a flex item as it is the child of the *#splash*. Normally flexboxs are rows, meaning content is laid out horizontally. We want the text to appear in the vertical center so we can convert the flex direction to be a column, rather than a row. We then use *justify-content: center* to place the <h1> / flex item in the vertical center.

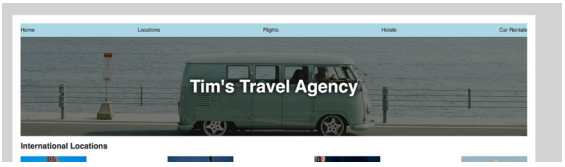


Save and preview, your webpage should look something like the image on the right.

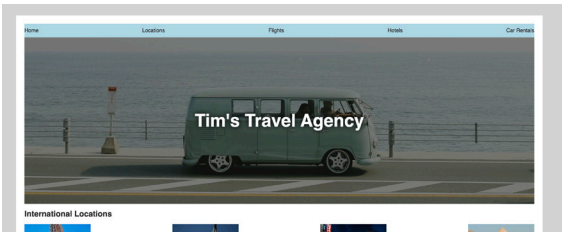


Our text is in the vertical center, but we also want it in the horizontal center. We can do this by adding the CSS property *align-items*. The align-items CSS property lays out flex items perpendicular to the direction of the flexbox. In otherwords, for a flex row, *align-items* places items vertically, for a flex column *align-items* places items horizontally. Like with justify-content we can use *start*, *end* and *center* to place the text. Place it in the *center*.

Save and your splash text should be in the vertical and horizontal center.



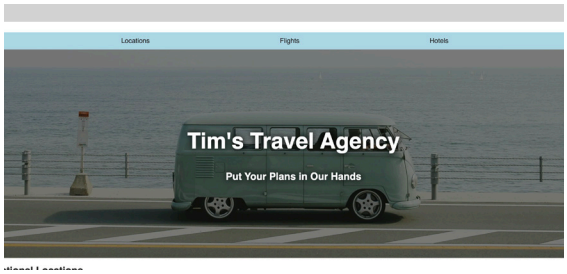
After showing Tim, he decides he wants the splash area to be taller. He tells you to change the height of the *#splash* area from *300px* to *500px*. Save and preview again and you'll see that the text still remains in the vertical and horizontal center.



After looking at the splash area again Tim decides he wanted to add a tagline to the splash area as well, but still keep the lines in the vertical and horizontal center. In the HTML uncomment the <h2> that reads *"Put Your Plans in Our Hands."* Save and

© David Hurwich

preview in a browser, you will see that using flexbox all of the text remains in the vertical and horizontal center.

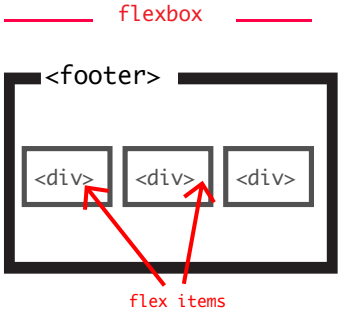


Flex Basis

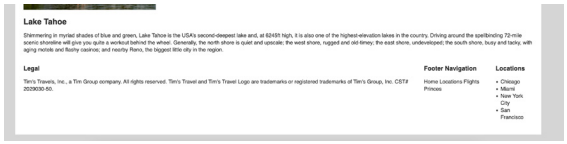
Let's convert the *<footer>* section in a three column layout. Add the following code to your CSS document.

```
footer {
   display: flex;
}
```

This makes the *<footer>* into a flexbox and each *<section>* inside becomes a column.



Unlike with previous uses of flexboxes our flex items are quite large due to the amount of text. The result is when one flex item has much more content than flex item, that particular flex item will be given more space. Since the Legal section had more content than the *Footer Navigation* or *Locations* it was given signifigant more space.

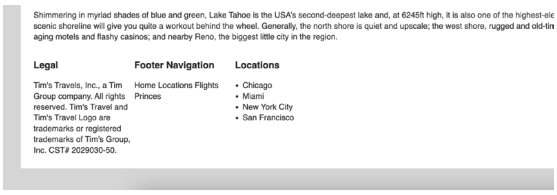


What if we wanted all three columns to have similar width or space, what could we do? The answer is the CSS property *flex-basis*. Flex basis can be applied to flex items themselves, not the flexbox and are able to set a specific width. Let's make each of the divs in the <footer> 200px wide. We can do that by adding code like this:

```
footer div {
    flex-basis: 200px;
}
```



This CSS rule will apply to all of the flex items in the *<footer>* and state that regardless of content the width of the item should be *200px*. Save and preview and you'll see that each section now becomes 200px wide.



If you resize the webpage you'll notice that the columns stay the same width. As the webpage gets wider the issue becomes that there is a lot of empty space. What if we want the columns to each equal width, but also expand and contract with the space? The answer is setting the flex-basis to 100%. When you do this, this tells the browser that it should take the total space and divide it evenly based on the number of flex items, in this case 3.



Save and preview you now you will see that each column gets the same width.

© David Hurwich

Flex Wrap

Finally, Tim would like to have US travel locations shown on the websit as well. For each city he wants a photo, the name of the city and some text describing the city.

Find *#national-layout* and make it a flexbox.

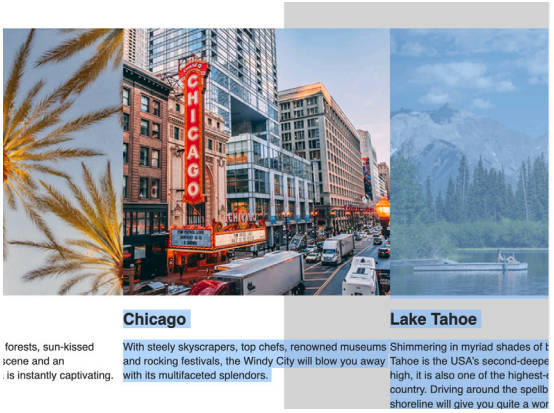
```
#national-layout {
   display: flex;
}
```



This turns the *#national-layout* div into a flexbox and each *<section>* tag becomes a flex item. Save and preview in a browser.

You will notice that our content breaks out of the webpage, why is this? Defaultly flexbox will take all of it's children / flex items and try to lay them out in one row. This becomes an issue when each flex item has a lot of content, such as the image text. There is literally not enough space horizontally to fit everything so it breaks out the *.container*.

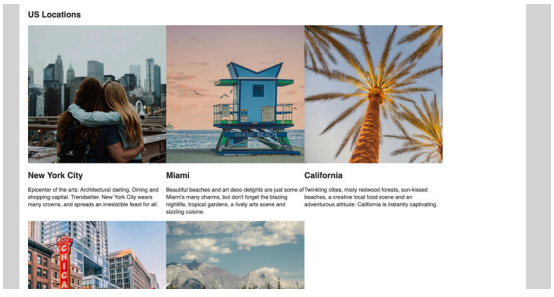
This practice is not good and does not look good visually, so what can we do? The answer is *flex-wrap*. Like it sounds like, this property allows your items to wrap or go to the next line. Set *flex-wrap* equal to *wrap*. Save and preview.



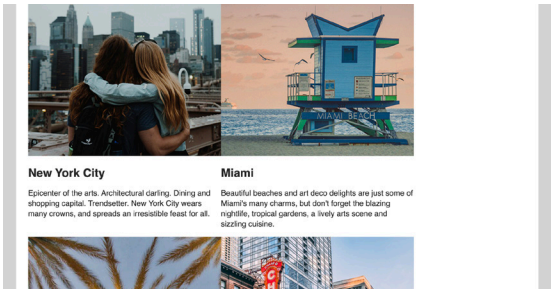
You may notice that each travel destination is now on its own line, why is this? The answer is, due to the amount of content in each flex item, each item was wrapping or going to the next line. How do we get multiple columns horizontally then? The answer is we need to make each column smaller than the total width, or use flex-basis.

Set each <section> inside the *#national-layout* to a specific size, like 300px. It will make each item 300px wide and then the flexbox will stack each flex item one after each other until it reaches the end of the flexbox, at which case it will wrap to the next line.

```
#national-layout section {
    flex-basis: 300px;
}
```



Save and preview. If you open your webpage wide you should see you get about 3 columns before it wraps to the next line. If you make the webpage thinner you can see that it breaks to become two and eventually one column for very thin webpages. This method of layout can be very useful when you want your webpage to be responsive, or adapt to devices of many sizes.



© David Hurwich

Flex-Grow

Now that the columns wrap you may notice that there are times where there is empty space in our design that is not being utilized. There is a solution to this called *flex-grow*. The *flex-grow* property allows flex items to grow in size **if** there is available space. If there is not space they will not grow, but if there is avaiable space it will be divided between all of the elements that are set to grow.

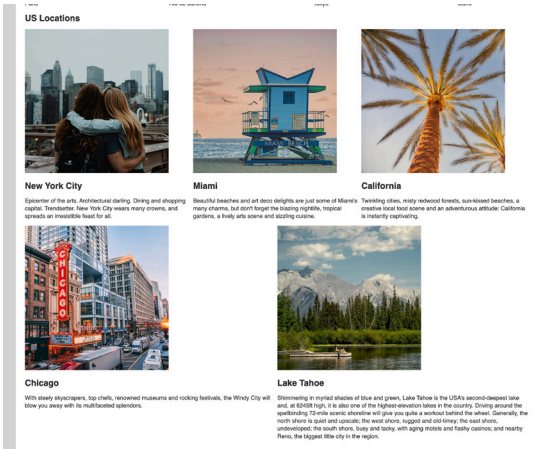
Return to *#destination-layout* section and add the following code:

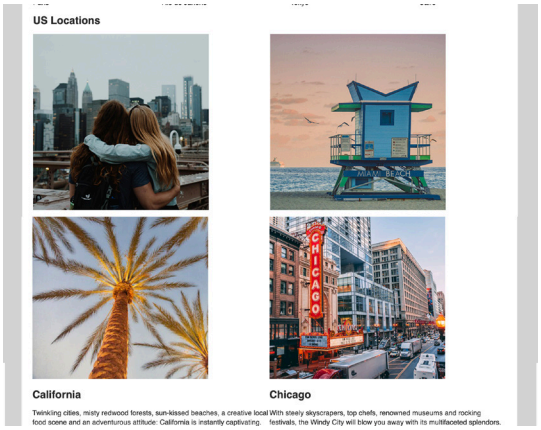
```
#national-layout section {
    flex-basis: 300px;
    flex-grow: 1;
}
```

What *flex-grow* with a property of *"1"* does is turn *"on"* the grow property. If the item was growing and we wanted it to not grow we would set *flex-grow* to *"0"* or *"off"*.

With each flex item growing the columns will expand to take advantage of the available space.

Save and preview. You should see that columns take up all the available horizontal space. Resize the browser larger and smaller and you will see that this continues even as wrapping is occuring.





Submission

Great work! We used flexbox to help us create a sophisticated layout with columns, centered splash areas and columns that adjusted based on available space.

Zip the lab folder and submit it to Canvas for grading.

© David Hurwich